Capítulo 11 - Agente de Monitoramento (Monitoring Agent)

11.1. Introdução ao Agente de Monitoramento

O **Agente de Monitoramento** é o componente de acompanhamento do Projeto Quimera. Ele **monitora conversas ativas** para detectar respostas, analisar sentimento e classificar o nível de interesse, permitindo que o usuário saiba quando e como responder.

11.1.1. Responsabilidades do Agente de Monitoramento

1. **Monitorar caixa de entrada:**
   - Verificar periodicamente mensagens não lidas
   - Detectar novas respostas
2. **Coletar mensagens:**
   - Extrair texto da resposta
   - Capturar timestamp
   - Identificar remetente
3. **Analisar resposta:**
   - Análise de sentimento (positivo, neutro, negativo)
   - Classificação de interesse (alto, médio, baixo)
   - Detecção de intenção (pergunta, afirmação, rejeição)
4. **Atualizar conversa:**
   - Registrar resposta no banco de dados
   - Atualizar status da conversa
   - Calcular tempo de resposta
5. **Notificar usuário:**
   - Emitir evento RespostaRecebida
   - Enviar notificação (se configurado)
   - Priorizar respostas de alto interesse

11.1.2. Desafios Principais

1. Polling vs. Push

Instagram não oferece webhooks. Precisamos fazer polling (verificar periodicamente), o que consome recursos.

2. Detecção de Mudanças

Identificar quais conversas têm novas mensagens sem carregar todas.

3. Análise de Contexto

Uma resposta curta como "oi" pode ser positiva ou negativa dependendo do contexto.

4. Rate Limiting

Verificar mensagens também conta para limites de requisições.

11.2. Arquitetura do Agente de Monitoramento

11.2.1. Componentes Principais

Plain Text

```
┌──────────────────────────────────────────────────────────┐
│        AGENTE DE MONITORAMENTO                           │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────────────────────────────┐               │
│   │  1. SCHEDULER                        │               │
│   │  - Executa verificação periódica     │               │
│   │  - Intervalo: 5-10 minutos           │               │
│   │  - Prioriza conversas recentes       │               │
│   └──────────────────────────────────────┘               │
│                    │                                      │
│                    ▼                                      │
│   ┌──────────────────────────────────────┐               │
│   │  2. COLETOR DE MENSAGENS             │               │
│   │  - Abre caixa de entrada             │               │
│   │  - Identifica conversas não lidas    │               │
│   │  - Extrai mensagens                  │               │
│   └──────────────────────────────────────┘               │
│                    │                                      │
│                    ▼                                      │
│   ┌──────────────────────────────────────┐               │
│   │  3. ANALISADOR DE SENTIMENTO         │               │
│   │  - Classifica emoção (NLP)           │               │
│   │  - Positivo/Neutro/Negativo          │               │
│   │  - Confiança da classificação        │               │
│   └──────────────────────────────────────┘               │
│                    │                                      │
```

```
│                    ▼                                    │
│   ┌──────────────────────────────────┐                 │
│   │  4. CLASSIFICADOR DE INTERESSE    │                 │
│   │  - Alto/Médio/Baixo               │                 │
│   │  - Baseado em múltiplos fatores   │                 │
│   │  - Score de 0-100                 │                 │
│   └──────────────────────────────────┘                 │
│                    │                                    │
│                    ▼                                    │
│   ┌──────────────────────────────────┐                 │
│   │  5. DETECTOR DE INTENÇÃO          │                 │
│   │  - Pergunta                       │                 │
│   │  - Afirmação                      │                 │
│   │  - Rejeição                       │                 │
│   │  - Interesse em encontro          │                 │
│   └──────────────────────────────────┘                 │
│                    │                                    │
│                    ▼                                    │
│   ┌──────────────────────────────────┐                 │
│   │  6. PERSISTÊNCIA                  │                 │
│   │  - Salva resposta no BD           │                 │
│   │  - Atualiza status da conversa    │                 │
│   │  - Emite evento RespostaRecebida  │                 │
│   └──────────────────────────────────┘                 │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

11.2.2. Fluxo de Monitoramento

Plain Text

```
INÍCIO (a cada 5-10 minutos)
  │
  ├─> 1. OBTER CONVERSAS ATIVAS
  │     ├─> Buscar conversas com status 'aguardando_resposta'
  │     ├─> Ordenar por data de envio (mais recentes primeiro)
  │     └─> Limitar a 50 conversas por execução
  │
  ├─> 2. PARA CADA CONVERSA:
  │     │
  │     ├─> 2.1. ABRIR CONVERSA
  │     │     ├─> Navegar até URL da conversa
  │     │     ├─> Aguardar carregamento (2-3s)
  │     │     └─> Verificar se há mensagens não lidas
  │     │
  │     ├─> 2.2. SE HÁ NOVA MENSAGEM:
  │     │     │
```

```
   |      |      ├─> Extrair texto da mensagem
   |      |      ├─> Capturar timestamp
   |      |      ├─> Capturar mídia (se houver)
   |      |      |
   |      |      ├─> 2.3. ANALISAR SENTIMENTO
   |      |      |      ├─> Usar modelo NLP (XLM-RoBERTa)
   |      |      |      ├─> Classificar: positivo/neutro/negativo
   |      |      |      └─> Obter score de confiança
   |      |      |
   |      |      ├─> 2.4. CLASSIFICAR INTERESSE
   |      |      |      ├─> Analisar comprimento da resposta
   |      |      |      ├─> Analisar tempo de resposta
   |      |      |      ├─> Analisar sentimento
   |      |      |      ├─> Analisar presença de perguntas
   |      |      |      ├─> Analisar emojis
   |      |      |      └─> Calcular score (0-100)
   |      |      |
   |      |      ├─> 2.5. DETECTAR INTENÇÃO
   |      |      |      ├─> Pergunta? (contém '?')
   |      |      |      ├─> Rejeição? (palavras-chave negativas)
   |      |      |      ├─> Interesse em encontro? (palavras-chave)
   |      |      |      └─> Afirmação genérica?
   |      |      |
   |      |      ├─> 2.6. SALVAR RESPOSTA
   |      |      |      ├─> INSERT na tabela 'mensagens'
   |      |      |      ├─> UPDATE status da conversa
   |      |      |      ├─> Calcular tempo_resposta
   |      |      |      └─> Incrementar contador de mensagens
   |      |      |
   |      |      └─> 2.7. EMITIR EVENTO
   |      |            ├─> Publicar 'RespostaRecebida' no Redis
   |      |            ├─> Incluir: perfil_id, sentimento, interesse
   |      |            └─> Notificar usuário (se alto interesse)
   |      |
   |      └─> 2.8. DELAY
   |            └─> Aguardar 5-10 segundos antes da próxima
   |
FIM
```

11.3. Coleta de Mensagens

11.3.1. Navegação até Caixa de Entrada

Python

```python
def abrir_caixa_entrada(driver):
    """Abre caixa de entrada do Instagram."""
    driver.get("https://www.instagram.com/direct/inbox/")
    time.sleep(random.uniform(2, 4))

    # Verificar se carregou
    try:
        driver.find_element(By.XPATH, "//div[@role='listbox']")
        return True
    except:
        return False


def obter_conversas_nao_lidas(driver):
    """
    Obtém lista de conversas com mensagens não lidas.

    Returns:
        list: Lista de elementos de conversa
    """
    # Conversas não lidas têm indicador visual específico
    conversas_nao_lidas = driver.find_elements(
        By.XPATH,
        "//div[contains(@class, 'x1n2onr6')]//div[contains(@style, 'font-
weight: 600')]"
    )

    return conversas_nao_lidas
```

11.3.2. Extração de Mensagens

Python

```python
def extrair_mensagem(driver, conversa_id):
    """
    Extrai última mensagem de uma conversa.

    Returns:
        dict: Dados da mensagem
    """
    try:
        # Localizar container de mensagens
        mensagens = driver.find_elements(
            By.XPATH,
            "//div[@role='row']//div[contains(@class, 'x1n2onr6')]"
        )
```

```python
        if not mensagens:
            return None

        # Última mensagem (do destinatário, não nossa)
        ultima_mensagem = mensagens[-1]

        # Extrair texto
        texto = ultima_mensagem.text

        # Extrair timestamp (se disponível)
        try:
            timestamp_elem = ultima_mensagem.find_element(By.XPATH,
".//time")
            timestamp = timestamp_elem.get_attribute("datetime")
        except:
            timestamp = None

        # Verificar se há mídia (foto, vídeo)
        tem_midia = False
        try:
            ultima_mensagem.find_element(By.XPATH, ".//img | .//video")
            tem_midia = True
        except:
            pass

        return {
            'texto': texto,
            'timestamp': timestamp,
            'tem_midia': tem_midia,
            'conversa_id': conversa_id
        }

    except Exception as e:
        print(f"Erro ao extrair mensagem: {e}")
        return None
```

11.4. Análise de Sentimento

11.4.1. Modelo de Sentimento

Usamos **XLM-RoBERTa** treinado para análise de sentimento multilíngue.

Python

```python
from transformers import pipeline

# Inicializar modelo (uma vez)
analisador_sentimento = pipeline(
    "sentiment-analysis",
    model="cardiffnlp/twitter-xlm-roberta-base-sentiment",
    tokenizer="cardiffnlp/twitter-xlm-roberta-base-sentiment"
)

def analisar_sentimento(texto):
    """
    Analisa sentimento de um texto.

    Returns:
        dict: Sentimento e confiança
    """
    resultado = analisador_sentimento(texto)[0]

    # Mapear labels
    mapeamento = {
        'LABEL_0': 'negativo',
        'LABEL_1': 'neutro',
        'LABEL_2': 'positivo'
    }

    sentimento = mapeamento.get(resultado['label'], 'neutro')
    confianca = resultado['score']

    return {
        'sentimento': sentimento,
        'confianca': confianca
    }


# Exemplo de uso
texto = "Adorei sua mensagem! Vamos sim conversar mais!"
resultado = analisar_sentimento(texto)
# {'sentimento': 'positivo', 'confianca': 0.98}
```

11.4.2. Análise Contextual

Sentimento pode variar com contexto. Exemplos:

Python

```python
def analisar_sentimento_contextual(texto, contexto_conversa):
    """
```

```
    Analisa sentimento considerando contexto da conversa.

    Args:
        texto: Texto da mensagem
        contexto_conversa: Mensagens anteriores
    """
    # Sentimento base
    sentimento_base = analisar_sentimento(texto)

    # Ajustar baseado em contexto
    if sentimento_base['sentimento'] == 'neutro':
        # "oi" sozinho é neutro, mas após pergunta interessante é positivo
        if contexto_conversa.get('ultima_mensagem_foi_pergunta'):
            if len(texto.strip()) < 20:  # Resposta curta
                sentimento_base['sentimento'] = 'positivo'
                sentimento_base['confianca'] = 0.7

    return sentimento_base
```

11.5. Classificação de Interesse

11.5.1. Fatores de Interesse

Interesse é calculado baseado em múltiplos fatores:

1. **Comprimento da resposta** (mais longa = mais interesse)
2. **Tempo de resposta** (mais rápido = mais interesse)
3. **Sentimento** (positivo = mais interesse)
4. **Presença de perguntas** (pergunta = mais interesse)
5. **Emojis** (mais emojis = mais interesse)
6. **Menção a encontro** (interesse explícito)

11.5.2. Implementação do Classificador

Python

```python
def classificar_interesse(mensagem, conversa):
    """
    Classifica nível de interesse de uma resposta.

    Returns:
        dict: Nível e score (0-100)
```

```
    """
    score = 0

    # 1. Comprimento da resposta (0-20 pontos)
    comprimento = len(mensagem['texto'])
    if comprimento > 100:
        score += 20
    elif comprimento > 50:
        score += 15
    elif comprimento > 20:
        score += 10
    else:
        score += 5

    # 2. Tempo de resposta (0-20 pontos)
    tempo_resposta = calcular_tempo_resposta(mensagem, conversa)
    if tempo_resposta < 300:  # < 5 minutos
        score += 20
    elif tempo_resposta < 3600:  # < 1 hora
        score += 15
    elif tempo_resposta < 86400:  # < 1 dia
        score += 10
    else:
        score += 5

    # 3. Sentimento (0-25 pontos)
    sentimento = analisar_sentimento(mensagem['texto'])
    if sentimento['sentimento'] == 'positivo':
        score += 25
    elif sentimento['sentimento'] == 'neutro':
        score += 12
    else:
        score += 0

    # 4. Presença de perguntas (0-15 pontos)
    if '?' in mensagem['texto']:
        score += 15

    # 5. Emojis (0-10 pontos)
    num_emojis = contar_emojis(mensagem['texto'])
    score += min(num_emojis * 2, 10)

    # 6. Menção a encontro (0-10 pontos)
    palavras_encontro = ['encontrar', 'conhecer', 'café', 'sair', 'vamos']
    if any(palavra in mensagem['texto'].lower() for palavra in
palavras_encontro):
        score += 10
```

```python
    # Classificar nível
    if score >= 70:
        nivel = 'alto'
    elif score >= 40:
        nivel = 'medio'
    else:
        nivel = 'baixo'

    return {
        'nivel': nivel,
        'score': score,
        'fatores': {
            'comprimento': comprimento,
            'tempo_resposta': tempo_resposta,
            'sentimento': sentimento['sentimento'],
            'tem_pergunta': '?' in mensagem['texto'],
            'num_emojis': num_emojis
        }
    }


def calcular_tempo_resposta(mensagem, conversa):
    """Calcula tempo entre envio e resposta (em segundos)."""
    timestamp_envio = conversa['timestamp_ultima_mensagem_enviada']
    timestamp_resposta = mensagem['timestamp']

    return (timestamp_resposta - timestamp_envio).total_seconds()


def contar_emojis(texto):
    """Conta número de emojis em um texto."""
    import emoji
    return emoji.emoji_count(texto)
```

11.6. Detecção de Intenção

11.6.1. Tipos de Intenção

Python

```python
class TipoIntencao:
    PERGUNTA = "pergunta"
    AFIRMACAO = "afirmacao"
    REJEICAO = "rejeicao"
```

```python
    INTERESSE_ENCONTRO = "interesse_encontro"
    CONTINUACAO = "continuacao"
```

11.6.2. Detector de Intenção

Python

```python
def detectar_intencao(texto):
    """
    Detecta intenção da mensagem.

    Returns:
        str: Tipo de intenção
    """
    texto_lower = texto.lower()

    # 1. Rejeição (prioridade alta)
    palavras_rejeicao = [
        'não', 'nao', 'desculpa', 'não estou interessada',
        'tenho namorado', 'não quero', 'para de me mandar'
    ]
    if any(palavra in texto_lower for palavra in palavras_rejeicao):
        return TipoIntencao.REJEICAO

    # 2. Interesse em encontro
    palavras_encontro = [
        'vamos', 'que tal', 'quando', 'onde', 'café',
        'encontrar', 'conhecer', 'sair', 'marcar'
    ]
    if any(palavra in texto_lower for palavra in palavras_encontro):
        return TipoIntencao.INTERESSE_ENCONTRO

    # 3. Pergunta
    if '?' in texto:
        return TipoIntencao.PERGUNTA

    # 4. Continuação (resposta que mantém conversa)
    palavras_continuacao = [
        'também', 'legal', 'interessante', 'conta mais',
        'que legal', 'adorei', 'que massa'
    ]
    if any(palavra in texto_lower for palavra in palavras_continuacao):
        return TipoIntencao.CONTINUACAO

    # 5. Afirmação genérica (padrão)
    return TipoIntencao.AFIRMACAO
```

```python
# Exemplos
detectar_intencao("Que legal! Você curte o quê?")  # PERGUNTA
detectar_intencao("Vamos tomar um café?")  # INTERESSE_ENCONTRO
detectar_intencao("Desculpa, não estou interessada")  # REJEICAO
detectar_intencao("Também adoro viajar!")  # CONTINUACAO
detectar_intencao("Oi")  # AFIRMACAO
```

11.7. Gerenciamento de Conversas

11.7.1. Estados da Conversa

Python

```python
class EstadoConversa:
    AGUARDANDO_RESPOSTA = "aguardando_resposta"
    RESPONDIDA = "respondida"
    EM_ANDAMENTO = "em_andamento"
    REJEITADA = "rejeitada"
    CONVERTIDA = "convertida"  # Obteve contato/encontro
    ABANDONADA = "abandonada"
```

11.7.2. Atualização de Status

Python

```python
def atualizar_conversa(conversa_id, resposta, classificacao):
    """
    Atualiza status da conversa após receber resposta.

    Args:
        conversa_id: ID da conversa
        resposta: Dados da resposta
        classificacao: Resultado da classificação
    """
    # Determinar novo status
    if classificacao['intencao'] == TipoIntencao.REJEICAO:
        novo_status = EstadoConversa.REJEITADA
    elif classificacao['intencao'] == TipoIntencao.INTERESSE_ENCONTRO:
        novo_status = EstadoConversa.CONVERTIDA
    elif classificacao['interesse']['nivel'] in ['alto', 'medio']:
        novo_status = EstadoConversa.EM_ANDAMENTO
    else:
        novo_status = EstadoConversa.RESPONDIDA
```

```
    # Atualizar no banco de dados
    query = """
        UPDATE conversas
        SET
            status = %s,
            ultima_resposta_em = %s,
            tempo_resposta_segundos = %s,
            nivel_interesse = %s,
            score_interesse = %s,
            sentimento_ultima_resposta = %s,
            intencao_ultima_resposta = %s,
            num_mensagens = num_mensagens + 1
        WHERE id = %s
    """

    cursor.execute(query, (
        novo_status,
        resposta['timestamp'],
        classificacao['interesse']['fatores']['tempo_resposta'],
        classificacao['interesse']['nivel'],
        classificacao['interesse']['score'],
        classificacao['sentimento']['sentimento'],
        classificacao['intencao'],
        conversa_id
    ))

    conn.commit()
```

11.8. Notificações

11.8.1. Sistema de Notificações

Python

```python
def notificar_usuario(conversa, resposta, classificacao):
    """
    Notifica usuário sobre resposta recebida.

    Notifica apenas se:
    - Interesse alto
    - Interesse em encontro
    - Rejeição (para encerrar conversa)
    """
    deve_notificar = (
```

```
        classificacao['interesse']['nivel'] == 'alto' or
        classificacao['intencao'] == TipoIntencao.INTERESSE_ENCONTRO or
        classificacao['intencao'] == TipoIntencao.REJEICAO
    )

    if not deve_notificar:
        return

    # Construir mensagem de notificação
    perfil = obter_perfil(conversa['perfil_id'])

    mensagem_notificacao = {
        'tipo': 'resposta_recebida',
        'perfil': {
            'nome': perfil['nome_completo'],
            'username': perfil['username'],
            'foto': perfil['foto_perfil_url']
        },
        'resposta': resposta['texto'],
        'classificacao': {
            'interesse': classificacao['interesse']['nivel'],
            'score': classificacao['interesse']['score'],
            'sentimento': classificacao['sentimento']['sentimento'],
            'intencao': classificacao['intencao']
        },
        'link_conversa':
f"https://www.instagram.com/direct/t/{conversa['thread_id']}"
    }

    # Enviar notificação (múltiplos canais)
    enviar_notificacao_push(mensagem_notificacao)
    enviar_notificacao_email(mensagem_notificacao)
    enviar_notificacao_telegram(mensagem_notificacao)


def enviar_notificacao_push(mensagem):
    """Envia notificação push (web/mobile)."""
    # Implementação com Firebase Cloud Messaging ou similar
    pass


def enviar_notificacao_email(mensagem):
    """Envia notificação por email."""
    # Implementação com SendGrid ou similar
    pass


def enviar_notificacao_telegram(mensagem):
```

```
    """Envia notificação via Telegram Bot."""
    import requests

    token = os.getenv("TELEGRAM_BOT_TOKEN")
    chat_id = os.getenv("TELEGRAM_CHAT_ID")

    texto = f"""
🔔 Nova Resposta - Interesse {mensagem['classificacao']['interesse'].upper()}

👤 {mensagem['perfil']['nome']} (@{mensagem['perfil']['username']})

💬 "{mensagem['resposta']}"

📊 Score: {mensagem['classificacao']['score']}/100
😊 Sentimento: {mensagem['classificacao']['sentimento']}
🎯 Intenção: {mensagem['classificacao']['intencao']}

🔗 {mensagem['link_conversa']}
"""

    requests.post(
        f"https://api.telegram.org/bot{token}/sendMessage",
        json={'chat_id': chat_id, 'text': texto}
    )
```

11.9. Otimizações

11.9.1. Polling Inteligente

Não verificar todas as conversas sempre. Priorizar conversas recentes.

Python

```
def obter_conversas_para_monitorar():
    """
    Obtém conversas que devem ser monitoradas.

    Prioriza:
    - Conversas mais recentes
    - Conversas com alto score de perfil
    - Conversas que ainda não receberam resposta
    """
    query = """
        SELECT * FROM conversas
        WHERE status = 'aguardando_resposta'
        AND ultima_mensagem_enviada_em > NOW() - INTERVAL '7 days'
```

```
        ORDER BY
            pontuacao_perfil DESC,
            ultima_mensagem_enviada_em DESC
        LIMIT 50
    """

    return executar_query(query)
```

11.9.2. Cache de Conversas Verificadas

Evitar verificar mesma conversa múltiplas vezes.

Python

```python
def marcar_conversa_verificada(conversa_id):
    """Marca conversa como verificada (cache Redis)."""
    redis_client.setex(
        f"conversa_verificada:{conversa_id}",
        300,  # TTL 5 minutos
        "1"
    )


def conversa_foi_verificada_recentemente(conversa_id):
    """Verifica se conversa já foi verificada recentemente."""
    return redis_client.exists(f"conversa_verificada:{conversa_id}")
```

11.10. Métricas e Monitoramento

11.10.1. Métricas Importantes

Python

```python
metricas_monitoramento = {
    'conversas_monitoradas': 150,
    'respostas_detectadas': 45,
    'taxa_resposta': 0.30,  # 30%
    'tempo_medio_resposta_horas': 6.5,
    'interesse_alto': 12,
    'interesse_medio': 20,
    'interesse_baixo': 13,
    'rejeicoes': 5,
    'conversoes': 3,  # Obteve encontro/contato
```

```
    'tempo_medio_verificacao_segundos': 8.2
}
```

11.10.2. Dashboard de Conversas

Python

```python
def gerar_dashboard_conversas():
    """Gera dashboard com estatísticas de conversas."""
    query = """
        SELECT
            status,
            COUNT(*) as total,
            AVG(tempo_resposta_segundos) as tempo_medio_resposta,
            AVG(score_interesse) as score_medio_interesse
        FROM conversas
        WHERE ultima_mensagem_enviada_em > NOW() - INTERVAL '30 days'
        GROUP BY status
    """

    resultados = executar_query(query)

    return {
        'por_status': resultados,
        'taxa_resposta_geral': calcular_taxa_resposta(),
        'taxa_conversao': calcular_taxa_conversao()
    }
```

11.11. Conclusão do Capítulo

Este capítulo cobriu o **Agente de Monitoramento** em profundidade:

- **Coleta de Mensagens:** Navegação, extração de texto e timestamp
- **Análise de Sentimento:** Modelo NLP, análise contextual
- **Classificação de Interesse:** 6 fatores, score 0-100, 3 níveis
- **Detecção de Intenção:** 5 tipos (pergunta, rejeição, encontro, etc.)
- **Gerenciamento de Conversas:** 6 estados, atualização automática
- **Notificações:** Push, email, Telegram
- **Otimizações:** Polling inteligente, cache
- **Métricas:** Taxa de resposta, tempo médio, conversões

No próximo capítulo, veremos o **Agente de Aprendizagem**, que usa feedback para melhorar o sistema continuamente.

Próximo Capítulo: Capítulo 12 - Agente de Aprendizagem

Referências:

- Sentiment Analysis with Transformers: https://huggingface.co/docs/transformers/tasks/sentiment_analysis
- Instagram Direct API (unofficial): https://github.com/ping/instagram_private_api
- Real-time Notifications: https://firebase.google.com/docs/cloud-messaging